

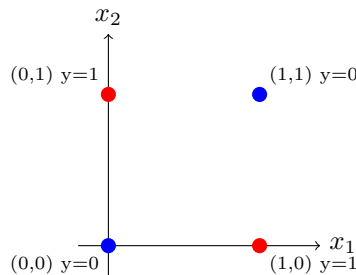
Lab Report – Neural Models: Learning, Depth, Activations, Output Layers

Apurva Verma

Task 1: Problem specification

$\mathcal{X} = \{0, 1\}^2$, $\mathcal{Y} = \{0, 1\}$. Labelled examples: $(0, 0) \rightarrow 0$, $(0, 1) \rightarrow 1$, $(1, 0) \rightarrow 1$, $(1, 1) \rightarrow 0$ (XOR: warning iff the sensors disagree).

Plot: in the x_1, x_2 plane the class-0 points sit at opposite corners $(0, 0)$, $(1, 1)$ and the class-1 points at the other corners $(0, 1)$, $(1, 0)$, so the two classes lie on the two diagonals of the unit square.



Linear separability: a line $w_1x_1 + w_2x_2 + b = 0$ would need $b < 0$, $w_1 + w_2 + b < 0$ (class 0) and $w_1 + b > 0$, $w_2 + b > 0$ (class 1). Adding the last two gives $w_1 + w_2 + 2b > 0$, but adding the first two gives $w_1 + w_2 + 2b < 0$: a contradiction, so no straight boundary works.

Prediction: a single affine map plus sigmoid can do no better than predicting ≈ 0.5 for every input (loss $\approx \ln 2$), getting at most 3 of 4 correct.

Task 2: Model design and validation criteria

Design: 2 inputs $\rightarrow$ 2 hidden units (tanh; sigmoid and ReLU tried later) $\rightarrow$ 1 output logit; sigmoid output, binary cross-entropy (implemented as `BCEWithLogitsLoss`), Adam, full batch.

1. **Why nonlinearity:** affine layers collapse to one affine map, which is a linear classifier and cannot represent XOR. The hidden nonlinearity lets the network re-represent the inputs in a space where the classes become linearly separable. Depth alone does not help; nonlinearity does. XOR tests exactly this claim about representation with four points.
2. **Why sigmoid + BCE:** the target is one yes/no answer; sigmoid maps the logit to a probability in $(0, 1)$, BCE is the negative log-likelihood of a Bernoulli label, and the logit gradient is simply $p - y$ (no saturation-induced vanishing from the loss).
3. **Success evidence:** (a) final loss far below $\ln 2 \approx 0.693$; (b) all four thresholded predictions correct; (c) first-layer gradient nonzero at an early step and finite-difference check agrees with autograd; (d) repeated-run behaviour over several seeds; (e) zero-init control fails, showing the learning is not trivial.

Hidden units get no targets: what each computes is determined only by the output-layer loss propagated backwards through the chain rule; backprop supplies each hidden unit a learning signal $\partial L / \partial h$.

Task 3: LLM prompts and generated code

The prompts below were used with LLM assistance to produce the code, which was inspected and run. Forward pass: `forward()`; scalar loss: `criterion(logits,y)`; reverse-mode AD: `loss.backward()`; parameter update: `optimizer.step()`.

Two changes made after the first run of the generated code: (1) a progress print of the loss every 500 steps, to see the loss decreasing rather than only the final number; (2) an `assert torch.isfinite(loss)` guard inside the training loop to catch NaN/inf immediately. Neither changes the architecture, data, loss or optimiser. The logged losses for seed 0 were 0.7152 (step 0), 0.00194 (500), 0.00064 (1000), 0.00032 (1500), 0.00019 (2000), 0.00012 (2500).

Prompt 1 (Task 3)

Generate minimal PyTorch code for the following model and dataset. Do not change the architecture or task.

Dataset (XOR, sensor-disagreement warning), all 4 examples, full batch:
`x = [[0,0],[0,1],[1,0],[1,1]], y = [0,1,1,0]`

Model: 2 inputs -> 2 hidden units (tanh activation) -> 1 output logit.
 Use `nn.Linear` layers with PyTorch's default random initialisation.
 Output: a single logit, trained with `BCEWithLogitsLoss` (mean reduction); apply sigmoid only to report probabilities.
 Optimiser: Adam, `lr=0.05`, full-batch training for 3000 CPU steps.
 Set `torch.manual_seed(0)` for reproducibility.

After training, report: the initial loss, the final loss, all four probabilities, the labels thresholded at 0.5, whether all 4 are correct, and t
 Add brief comments marking where the forward pass happens, where the scalar loss is formed, where `backward()` (reverse-mode AD) is invoked, and wh
 Save as `task3_xor.py` and run it. Explain each test in one sentence.

Prompt 2 (Task 4 A–C)

Using `task3_xor.py` without changing the architecture or task, create `task4_diagnostics.py` that does the following:

Part A: repeat the basic learning check for seeds 0 to 4 and report the initial loss, final loss and 4/4 correctness for each seed.
 Part B: for one forward+backward pass at initialisation, print `layer1.weight.grad`. Verify it against a finite-difference estimate of dL/dW_1 (eps=
 Part C: copy the experiment but set ALL weights and biases to zero before training (same architecture). Train and print the two rows of the hidden
 Run it and show the output.

Prompt 3 (Task 4 D)

Create `task4_activations.py`. Train the same random-initialised 2-2-1 XOR network three times, changing ONLY the hidden activation: sigmoid, tanh,
 For each run, record: the final loss, whether all 4 examples are classified correctly, and the Euclidean (L2) norm of the first-layer weight gradi
 Also print the hidden pre-activations and activations at the end, so we can tell sigmoid saturation apart from dead ReLU units.
 Because 4 data points and 1 seed are fragile, also repeat over seeds 0 to 9 and report how many of the 10 runs succeed for each activation.
 Print a result table with columns: Hidden activation | Final loss | 4/4 correct? | Early ||grad W1||_2.

Prompt 4 (Task 5)

Modify ONLY the output/loss portion of `task3_xor.py` to make `task5_multiclass.py`:
 Classes: 0 = (0,0) both inactive, 1 = (0,1) or (1,0) disagree, 2 = (1,1) both active.
 Keep the 2-unit tanh hidden layer. Replace the output with `nn.Linear(2, 3)` producing 3 logits, trained with `nn.CrossEntropyLoss`.
 Before training, print the shape of the final weight matrix and the logits shape per example.
 After training, print the softmax probability vectors for all four inputs and the predicted classes, and verify that one probability vector sums
 Verify numerically that the gradient of the loss with respect to the logits equals $(p - y_{\text{onehot}})/N$.
 Optional diagnostic: add 100 to all three logits and show the softmax output is unchanged; then show that a naive `exp()` overflows with a large sh

Task 4: Results

Part A (basic learning check)

Seed 0: initial loss 0.7152, final 0.000083; probabilities [5.9e-5, 0.99990, 0.99988, 5.1e-5]; labels [0, 1, 1, 0]; 4/4 correct. Seeds 0–4: seeds 0, 2, 3, 4 succeed (final loss $\approx 10^{-4}$); seed 1 stalls at loss 0.3466 with 1 example wrong (a poor local minimum), so success is not guaranteed for a 2-unit hidden layer.

Part B (backprop check)

`layer1.weight.grad` is $\partial L / \partial W^{(1)}$, the derivative of the scalar mean loss w.r.t. each first-layer weight. At init (seed 0) autograd gives `[[0.0005, 0.0006], [-0.0426, -0.0448]]`; central finite differences ($\epsilon = 10^{-4}$) give `[[0.0009, 0.0009], [-0.0423, -0.0447]]`, max abs difference 3.9×10^{-4} (float32 roundoff). The mean-loss gradient equals the average of the four per-example gradients (verified, `True`) because differentiation is linear:

$$\nabla \frac{1}{N} \sum_i L_i = \frac{1}{N} \sum_i \nabla L_i.$$

Part C (zero-initialisation symmetry)

The two rows of $W^{(1)}$ are identical at steps 0, 1, 2, 5, 10, 100 and 3000 (in fact they stay exactly $[0, 0]$). Final loss $0.6931 = \ln 2$, all probabilities 0.5, not 4/4 correct. Identical hidden units compute the same output and, since the output weights are also identical (zero), receive the same gradient, so every update keeps them identical; symmetry is never broken. (With all-zero weights, tanh gives zero hidden activations, so the output-weight gradient is also zero, and the first-layer gradient is zero because the output weights are zero: the network is also at a saddle point.)

Part D (activation experiment, seed 0, Adam lr 0.05, 3000 steps)

Hidden activation	Final loss	4/4 correct?	Early $\ \nabla_{W^{(1)}} L\ _2$	Seeds OK (of 10)
Sigmoid	0.4774	No	0.000891	4
Tanh	0.000083	Yes	0.061785	4
ReLU	0.6931	No	0.001699	3

Interpretation. For this single seed, tanh learned XOR while sigmoid and ReLU did not, and the tanh early gradient was much larger ($\sim 70\times$ sigmoid, $\sim 36\times$ ReLU). The hidden-layer inspection shows different mechanisms: the sigmoid run ended with large |pre-activations| (± 11 to ± 25) and activations pinned near 0/1, i.e. saturation with tiny derivatives; the ReLU run ended with all pre-activations negative and all activations exactly 0, i.e. dead units with zero gradient. However, across seeds 0–9 the success counts are 4/10, 4/10 and 3/10, so with four data points and a 2-unit hidden layer I do not conclude any activation is universally better; the seed-0 difference is largely about initial conditions. Saturation vs. dead ReLU is distinguished by pre-activations: large magnitude with activation near 0/1 (sigmoid) versus negative pre-activation with activation exactly 0 (ReLU).

Task 5: Three-class extension

Output changed to `nn.Linear(2,3)` with `CrossEntropyLoss`. Predictions made beforehand: final weight matrix shape 3×2 ; 3 logits per example; softmax sums to one because $\sum_k e^{z_k} / \sum_j e^{z_j} = 1$ by construction; logit gradient $p - y$ because $L = -\log p_y$ and $\partial L / \partial z_k = p_k - \mathbb{I}[k = y]$. Confirmed: weight shape $(3, 2)$, logits shape $(3,)$. Final loss 3.9×10^{-5} ; predicted classes $[0, 1, 1, 2]$ match targets. Probabilities:

```
(0,0): [9.9995e-01, 4.8949e-05, 7.3595e-09]
(0,1): [2.3397e-05, 9.9997e-01, 7.4549e-06]
(1,0): [2.3177e-05, 9.9997e-01, 7.0238e-06]
(1,1): [2.7497e-07, 4.5178e-05, 9.9995e-01]
```

For input (0,1) the probabilities sum to 1.0. The gradient w.r.t. logits equals $(p - y_{\text{onehot}})/N$ (**True**; the $1/N$ comes from the mean loss). Adding 100 to all logits leaves softmax unchanged (**True**), because the common factor e^{100} cancels. A naive `exp` with +1000 overflows to `nan`; subtracting the max logit first gives the correct result, which is why stable implementations do so.

Reflection Questions

1. Depth is not enough: stacking affine layers is still one affine map. The nonlinear activation is what lets a hidden layer make XOR separable; depth then composes nonlinear features.
2. The loss fell from 0.715 to 8×10^{-5} and all four predictions flipped to the right XOR labels, and the autograd gradient matched finite differences. The gradient therefore pointed in a useful descent direction, not merely a nonzero one. (The zero-init run had nonzero-looking machinery but loss stuck at $\ln 2$.)
3. With identical weights, both hidden units compute the same output and get the same gradient, so they remain identical forever; two units act as one and cannot form distinct features. Random init breaks the symmetry.
4. Scientific: sigmoid saturates (derivative ≤ 0.25 , near 0 when saturated), tanh has derivative up to 1 and is zero-centred, ReLU has derivative 1 when active but exactly 0 when the unit is negative. Engineering observation: early gradient norms were 0.00089 (sigmoid), 0.0618 (tanh), 0.0017 (ReLU) for seed 0, but success rates over 10 seeds were similar (4/4/3), so the gradient size at one step is not a reliable predictor alone.

5. The output activation and loss must match the target type and probabilistic model: sigmoid+BCE for one Bernoulli label, softmax+cross-entropy for one-of- K . Mismatches (e.g. softmax with BCE, or MSE on logits) give wrong likelihoods and worse gradients; the matched pair gives the clean $p - y$ logit gradient.
6. Productivity: the LLM produced the boilerplate and tests (finite differences, per-example gradients, stability demo) quickly. Human verification was essential for checking that the code implements the specified 2–2–1 experiment, for noticing seed 1 fails and that the 10-seed counts contradict a naive “tanh is best” reading of one run, and for interpreting the finite-difference gap as float32 roundoff.
7. Keep: loss/prediction checks, gradient-norm monitoring, NaN/overflow checks, symmetry-breaking init, multi-seed repeats, cheap per-layer activation statistics, and spot-checks of a few gradient entries. Too expensive at scale: exhaustive finite-difference gradient checks (cost $\propto$ number of parameters), and running many seeds of full training.

Code

task3_xor.py

```
import torch
import torch.nn as nn

torch.manual_seed(0)

# XOR dataset (sensor-disagreement warning), full batch
x = torch.tensor([[0., 0.], [0., 1.], [1., 0.], [1., 1.]])
y = torch.tensor([[0.], [1.], [1.], [0.]])

# 2 -> 2 (tanh) -> 1 logit, default PyTorch init
layer1 = nn.Linear(2, 2)
layer2 = nn.Linear(2, 1)
params = list(layer1.parameters()) + list(layer2.parameters())

criterion = nn.BCEWithLogitsLoss() # mean reduction
optimizer = torch.optim.Adam(params, lr=0.05)

def forward(inp):
    return layer2(torch.tanh(layer1(inp))) # forward pass -> logits

with torch.no_grad():
    initial_loss = criterion(forward(x), y).item()

for step in range(3000):
    optimizer.zero_grad()
    logits = forward(x) # forward pass
    loss = criterion(logits, y) # scalar loss formed here
    loss.backward() # reverse-mode AD
    optimizer.step() # parameters updated here
    assert torch.isfinite(loss), f"loss not finite at step {step}" # change 2: NaN/inf guard
    if step % 500 == 0: # change 1: log loss every 500 steps
        print(f"step {step:4d} loss {loss.item():.6f}")

# Gradient of first-layer weight after a fresh forward + backward
optimizer.zero_grad()
final_loss_t = criterion(forward(x), y)
final_loss_t.backward()
grad_w1 = layer1.weight.grad.clone()

with torch.no_grad():
    probs = torch.sigmoid(forward(x))
    labels = (probs >= 0.5).float()
    correct = bool((labels == y).all())

print(f"Initial loss: {initial_loss:.6f}") # test 1: loss started high (~ln2)
print(f"Final loss: {final_loss_t.item():.6f}") # test 2: loss decreased -> learning
print("Probabilities:", probs.squeeze().tolist()) # test 3: outputs near 0/1
print("Labels (>=0.5):", labels.squeeze().tolist()) # test 4: thresholded predictions
print("All 4 correct:", correct) # test 5: matches XOR targets
print("layer1.weight.grad =\n", grad_w1) # test 6: gradient tensor dL/dW1
```

task4_diagnostics.py

```
import torch
import torch.nn as nn

x = torch.tensor([[0., 0.], [0., 1.], [1., 0.], [1., 1.]])
y = torch.tensor([[0.], [1.], [1.], [0.]])
crit = nn.BCEWithLogitsLoss()
```

```

def make(seed=None, zero=False):
    if seed is not None:
        torch.manual_seed(seed)
    l1, l2 = nn.Linear(2, 2), nn.Linear(2, 1)
    if zero:
        for p in list(l1.parameters()) + list(l2.parameters()):
            nn.init.zeros_(p)
    return l1, l2

def fwd(l1, l2, inp):
    return l2(torch.tanh(l1(inp)))

def train(l1, l2, steps=3000, record=()):
    opt = torch.optim.Adam(list(l1.parameters()) + list(l2.parameters()), lr=0.05)
    snaps = {}
    if 0 in record:
        snaps[0] = l1.weight.detach().clone()
    for s in range(1, steps + 1):
        opt.zero_grad()
        crit(fwd(l1, l2, x), y).backward()
        opt.step()
        if s in record:
            snaps[s] = l1.weight.detach().clone()
    return snaps

def evaluate(l1, l2):
    with torch.no_grad():
        loss = crit(fwd(l1, l2, x), y).item()
        p = torch.sigmoid(fwd(l1, l2, x))
        ok = bool((p >= 0.5).float() == y).all()
    return loss, p.squeeze().tolist(), ok

print("=== Part A: seeds 0-4 ===")
for seed in range(5):
    l1, l2 = make(seed)
    with torch.no_grad():
        init = crit(fwd(l1, l2, x), y).item()
    train(l1, l2)
    fl, _, ok = evaluate(l1, l2)
    print(f"seed {seed}: initial {init:.4f} final {fl:.6f} 4/4 correct: {ok}")

print("\n=== Part B: gradient check (seed 0, at init) ===")
l1, l2 = make(0)
crit(fwd(l1, l2, x), y).backward()
g = l1.weight.grad.clone()
print("autograd layer1.weight.grad =\n", g)

eps = 1e-4
fd = torch.zeros_like(g)
with torch.no_grad():
    for i in range(2):
        for j in range(2):
            orig = l1.weight[i, j].item()
            l1.weight[i, j] = orig + eps
            lp = crit(fwd(l1, l2, x), y).item()
            l1.weight[i, j] = orig - eps
            lm = crit(fwd(l1, l2, x), y).item()
            l1.weight[i, j] = orig
            fd[i, j] = (lp - lm) / (2 * eps)
print("finite-difference =\n", fd)
print("max abs diff:", (g - fd).abs().max().item())

# mean-loss gradient == average of per-example gradients
per = []
for k in range(4):
    l1.zero_grad(); l2.zero_grad()
    crit(fwd(l1, l2, x[k:k + 1]), y[k:k + 1]).backward()
    per.append(l1.weight.grad.clone())
avg = torch.stack(per).mean(0)
print("average of 4 per-example grads =\n", avg)
print("equals mean-loss grad:", torch.allclose(avg, g, atol=1e-6))

print("\n=== Part C: all-zero initialisation ===")
l1, l2 = make(zero=True)
rec = (0, 1, 2, 5, 10, 100, 3000)
snaps = train(l1, l2, record=rec)
for s in rec:
    W = snaps[s]
    print(f"step {s:4d}: row0={W[0].tolist()} row1={W[1].tolist()} identical={torch.equal(W[0], W[1])}")
fl, p, ok = evaluate(l1, l2)
print(f"final loss {fl:.6f} probs {p} 4/4 correct: {ok}")

```

task4_activations.py

```
import torch
import torch.nn as nn

x = torch.tensor([[0., 0.], [0., 1.], [1., 0.], [1., 1.]])
y = torch.tensor([[0.], [1.], [1.], [0.]])
crit = nn.BCEWithLogitsLoss()
acts = {"Sigmoid": torch.sigmoid, "Tanh": torch.tanh, "ReLU": torch.relu}

def run(act, seed):
    torch.manual_seed(seed)
    l1, l2 = nn.Linear(2, 2), nn.Linear(2, 1)
    opt = torch.optim.Adam(list(l1.parameters()) + list(l2.parameters()), lr=0.05)
    early = None
    for s in range(1, 3001):
        opt.zero_grad()
        loss = crit(l2(act(l1(x))), y)
        loss.backward()
        if s == 1:
            early = l1.weight.grad.norm().item() # Euclidean (Frobenius) norm at step 1
        opt.step()
    with torch.no_grad():
        a = l1(x)
        h = act(a)
        fl = crit(l2(h), y).item()
        ok = bool((torch.sigmoid(l2(h)) >= 0.5).float() == y).all())
    return fl, ok, early, a, h

print("Hidden activation | Final loss | 4/4 correct? | Early ||grad W1||_2 (seed 0)")
details = {}
for n, f in acts.items():
    fl, ok, g, a, h = run(f, 0)
    details[n] = (a, h)
    print(f"{n:<17} | {fl:.6f} | {str(ok):<12} | {g:.6f}")

print("\nFinal hidden pre-activations / activations (seed 0):")
for n, (a, h) in details.items():
    print(f"--- {n}\npre-activation:\n{a}\nactivation:\n{h}")

print("\nSuccess counts over seeds 0-9:")
for n, f in acts.items():
    wins = sum(run(f, s)[1] for s in range(10))
    print(f"{n}: {wins}/10")
```

task5_multiclass.py

```
import torch
import torch.nn as nn

torch.manual_seed(0)

x = torch.tensor([[0., 0.], [0., 1.], [1., 0.], [1., 1.]])
y = torch.tensor([0, 1, 1, 2]) # class indices

layer1 = nn.Linear(2, 2)
layer2 = nn.Linear(2, 3) # 3 logits
crit = nn.CrossEntropyLoss()
opt = torch.optim.Adam(list(layer1.parameters()) + list(layer2.parameters()), lr=0.05)

def fwd(inp):
    return layer2(torch.tanh(layer1(inp)))

print("Final weight matrix shape:", tuple(layer2.weight.shape))
print("Logits shape per example:", tuple(fwd(x[:1]).shape[1:]))

for _ in range(3000):
    opt.zero_grad()
    crit(fwd(x), y).backward()
    opt.step()

with torch.no_grad():
    logits = fwd(x)
    p = torch.softmax(logits, dim=1)
    print("Final loss:", crit(logits, y).item())
    print("Softmax probabilities:\n", p)
    print("Predicted classes:", p.argmax(1).tolist(), " targets:", y.tolist())
    print("Example 1 probs:", p[1].tolist(), " sum =", p[1].sum().item())

# gradient wrt logits == (p - y_onehot)/N
z = fwd(x).detach().requires_grad_(True)
crit(z, y).backward()
expected = (torch.softmax(z.detach(), 1) - nn.functional.one_hot(y, 3).float()) / 4
print("dL/dlogits == (p - y)/N:", torch.allclose(z.grad, expected, atol=1e-7))
```

```
# shift invariance and stability
print("softmax(z+100) == softmax(z):", torch.allclose(torch.softmax(logits + 100, 1), p, atol=1e-6))
big = logits[0] + 1000
naive = torch.exp(big) / torch.exp(big).sum()
stable = torch.exp(big - big.max()) / torch.exp(big - big.max()).sum()
print("naive exp with +1000:", naive.tolist())
print("max-subtracted with +1000:", stable.tolist())
```